# Package 'golemenviro'

November 10, 2026

**Title** Real-Time Visualization of Pimoroni Enviro Sensor Data from Prometheus

**Description** A 'shiny' application, built as a 'golem' package, that displays the readings of Pimoroni Enviro and Enviro+ sensors exported to 'Prometheus': gauges with the latest readings, and a time series chart per sensor.

**Version** 0.0.0.9000

**License** GPL-3

**URL** https://sedzinfo.github.io/golemenviro/, https://github.com/sedzinfo/golemenviro, https://sedzinfo-golemenviro.share.connect.posit.cloud/

**BugReports** https://github.com/sedzinfo/golemenviro/issues

**Encoding** UTF-8

**Roxygen** list(markdown = TRUE)

**Depends** R (>= 4.1.0)

**Imports** config,
golem,
htmltools,
httr,
jsonlite,
plotly,
shiny,
shinydashboard,
stats,
zoo

**Suggests** testthat (>= 3.0.0)

**Config/testthat/edition** 3

**Author** Dimitrios Zacharatos [aut, cre]

**Maintainer** Dimitrios Zacharatos <dzacharatos@yahoo.com>

**Config/roxygen2/version** 8.1.1

# Contents

app_time_zone . . . 2
blue_steps . . . 3
comfort_summary . . . 3
get_daily . . . 4
get_data . . . 5
get_hourly . . . 6
metric_mid . . . 6
pimoroni . . . 7
plot_average_day . . . 8
plot_comfort . . . 9
plot_daily_means . . . 10
plot_daily_pattern . . . 10
plot_gauge . . . 11
plot_time_series . . . 12
red_steps . . . 13
symbol_mean . . . 13
who_guidelines . . . 14

**Index** **15**

---

`app_time_zone` *The time zone of the app*

---

## Description

The one `pimoroni()` sets while the app runs, for the axis titles of the hours.

## Usage

```
app_time_zone()
```

## Value

The name of the time zone, such as "Europe/Bucharest".

## Examples

```
app_time_zone()
```

## blue_steps *Steps of one blue, from light to dark*

### Description

The color scale of the daily pattern. In dark mode it runs the other way, so that low values fade into the dark page and high values stand out, as they do on a light page.

### Usage

```
blue_steps
```

### Format

A character vector of 5 hexadecimal colors.

### Examples

```
blue_steps
```

## comfort_summary *Share of hours in and around the comfort zone*

### Description

Pairs the hourly averages of temperature and humidity by hour and counts how many hours fall inside the comfortable ranges, and how many are too warm, too cool, too dry or too humid (an hour can be both too warm and too dry).

### Usage

```
comfort_summary(
  temperature,
  humidity,
  temperature_range = c(20, 25),
  humidity_range = c(40, 60)
)
```

### Arguments

`temperature`, `humidity`
: Data frames from `get_hourly()` of temperature and humidity.

`temperature_range`, `humidity_range`
: The comfortable ranges, the same as the green bands of the Current tab: 20 to 25 degrees Celsius and 40 to 60%.

## Value

A list: `hours`, the number of hours with both readings, and the percentages `comfortable`, `too_warm`, `too_cool`, `too_dry` and `too_humid`.

## Examples

```
hours <- expand.grid(hour = 0:23, date = as.Date("2026-10-01") + 0:6)
temperature <- transform(hours, value = 22 + 4 * sin((hour - 9) / 24 * 2 * pi))
humidity <- transform(hours, value = 45 - 8 * sin((hour - 9) / 24 * 2 * pi))
comfort_summary(temperature, humidity)
```

---

# `get_daily` *Get the daily averages of one sensor*

---

## Description

The average of the hourly averages of each whole day, midnight to midnight in the time zone of the app, of the `days` days before today, for the Air Quality tab. Today, still running, is left out, and so is a day with readings in fewer than 18 of its hours (75%), as in the usual rules for daily averages of air quality.

## Usage

```
get_daily(urlq, metric, days = 30, end_time = Sys.time())
```

## Arguments

| | |
|---|---|
| `urlq` | Prometheus query_range url. |
| `metric` | Name of the metric, one of `metrics`. |
| `days` | Number of days, up to the start of the current hour. |
| `end_time` | A time on the day after the last day, by default now. |

## Value

A data frame with one row per day: `date` (a Date), `value` (the average) and `hours` (how many hours had readings). It has no rows when no day has enough readings, and is `NULL`, with a message, when the server cannot be reached.

## Examples

```
## Not run:
url <- "http://pip1.crabdance.com:1507/api/v1/query_range"
get_daily(url, "PM25", days = 30)

## End(Not run)
```

get_data *Get Data from Prometheus*

## Description

Downloads the readings of all the sensors of a Pimoroni Enviro or Enviro+ from Prometheus, at about 1000 points in time between `start_time` and `end_time`. When these are an hour or more apart (a window of about 6 weeks or more), each point is the average of the readings since the previous one, so that the day and night cycle does not show up as noise; otherwise it is the reading at that time.

## Usage

```
get_data(
  start_time = format(Sys.time() - 60 * 60 * 24 * 7, "%Y-%m-%dT%H:%M:%SZ", tz =
    "UTC"),
  end_time = format(Sys.time(), "%Y-%m-%dT%H:%M:%SZ", tz = "UTC"),
  urlq
)
```

## Arguments

| | |
|---|---|
| `start_time` | start time, in UTC, in this format: "2025-01-31T11:48:08Z" |
| `end_time` | end time, in UTC, in this format: "2025-02-07T11:48:33Z" |
| `urlq` | Prometheus url |

## Value

A data frame with the Date and the Timestamp of the readings and one column per sensor, or `NULL`, with a message, when the server cannot be reached or sends no data. Sensors whose readings never change are left out, as the device does not have them.

## Examples

```
prometheus_url <- c(
  "http://pip1.crabdance.com:1507/api/v1/query_range",
  "http://pip1.crabdance.com:1505/api/v1/query_range"
)
start_time <- format(Sys.time() - 60 * 60 * 24 * 7, "%Y-%m-%dT%H:%M:%SZ", tz = "UTC")
end_time <- format(Sys.time(), "%Y-%m-%dT%H:%M:%SZ", tz = "UTC")
filtered_df <- get_data(start_time = start_time, end_time = end_time, urlq = prometheus_url[2])
head(filtered_df)
filtered_df <- get_data(start_time = start_time, end_time = end_time, urlq = prometheus_url[1])
head(filtered_df)
```

## get_hourly *Get the hourly averages of one sensor*

### Description

Asks Prometheus for the average of each clock hour of the last `days` days, for the Daily Pattern tab. The average Prometheus gives at a time covers the hour before it, so it is filed under the hour that starts one hour earlier, in the time zone of the app.

### Usage

```
get_hourly(urlq, metric, days = 30, end_time = Sys.time())
```

### Arguments

| | |
|---|---|
| `urlq` | Prometheus query_range url. |
| `metric` | Name of the metric, one of `metrics`. |
| `days` | Number of days, up to the start of the current hour. |
| `end_time` | The time to count back from, by default now. |

### Value

A data frame with one row per hour: `date` (a Date), `hour` (0 to 23) and `value`. It has no rows when the server has no readings of the metric in that time, and is `NULL`, with a message, when the server cannot be reached.

### Examples

```
## Not run:
url <- "http://pip1.crabdance.com:1507/api/v1/query_range"
get_hourly(url, "temperature", days = 7)

## End(Not run)
```

## metric_mid *The value colored white in the daily pattern of each metric*

### Description

By metric name, the value that `plot_daily_pattern()` colors white, with lower values in blue and higher in red: room temperature, the middle of the comfortable range of humidity (40 to 60%), the standard atmospheric pressure at sea level, and for particles the daily guideline of the World Health Organization (who_guidelines). `NA` for the metrics with no such value (light, proximity, the gas sensors, whose resistance is only relative, and PM1, which has no guideline): these keep blue_steps.

## Usage

```
metric_mid
```

## Format

A named numeric vector, in the units of the readings.

## Examples

```
metric_mid
metric_mid[["temperature"]]
```

---

`pimoroni` *Run the Pimoroni Enviro app*

---

## Description

Starts a Shiny app with the readings of Pimoroni Enviro and Enviro+ sensors exported to Prometheus: gauges with the latest readings in the Current tab, and a chart per sensor in the Time Series tab. The app has a dark mode, switched by the button at the right of the navbar.

## Usage

```
pimoroni(
  prometheus_url,
  timezone = "Europe/Bucharest",
  analytics = FALSE,
  onStart = NULL,
  options = list(),
  enableBookmarking = NULL,
  uiPattern = "/",
  ...
)
```

## Arguments

| | |
|---|---|
| `prometheus_url` | Character vector of Prometheus query_range urls, one per device, such as "http://pip1.crabdance.com:150 |
| `timezone` | The time zone the readings are shown in, while the app runs. |
| `analytics` | `TRUE` to add Google Analytics, through the cookie banner shared with the sites of sedzinfo.github.io, which loads it only after the visitor accepts. Meant for the public deployment (see `app.R`); off by default, so that running the app yourself sends nothing. |
| `onStart` | See `shiny::shinyApp()`. |
| `options` | See `shiny::shinyApp()`. |
| `enableBookmarking` | See `shiny::shinyApp()`. |
| `uiPattern` | See `shiny::shinyApp()`. |
| `...` | Further options passed to `golem::with_golem_options()`. |

## Value

A shiny::shinyApp() object. Printing it, or calling pimoroni() at the console, opens the app in the browser.

## Examples

```
if (interactive()) {
  pimoroni(prometheus_url = c(
    "http://pip1.crabdance.com:1507/api/v1/query_range",
    "http://pip1.crabdance.com:1505/api/v1/query_range"
  ))
}
```

---

# plot_average_day *A typical day of one sensor*

---

## Description

For each hour of the day, the median of the hourly averages over all the days (the line), and the middle half of the days, between the 25th and 75th percentiles (the band): a narrow band means that hour is much the same every day.

## Usage

```
plot_average_day(hourly, variable, unit = "", mode = "light")
```

## Arguments

| | |
|---|---|
| hourly | Data frame from get_hourly(). |
| variable | Name of the sensor, for the title and the hover labels. |
| unit | Unit of the readings, "" for none. |
| mode | "light" or "dark", the mode of the app. |

## Value

A plotly object.

## Examples

```
hourly <- expand.grid(hour = 0:23, date = as.Date("2026-10-01") + 0:13)
hourly$value <- 21 + 2 * sin((hourly$hour - 9) / 24 * 2 * pi) + rnorm(nrow(hourly), sd = 0.5)
plot_average_day(hourly, "Temperature", "C")
url <- "http://pip1.crabdance.com:1507/api/v1/query_range"
plot_average_day(get_hourly(url, "humidity", days = 30), "Humidity", "%")
plot_average_day(get_hourly(url, "humidity", days = 360), "Humidity", "%")
```

`plot_comfort` *Comfort zone: temperature against humidity*

## Description

A point for every hour, at its average temperature and humidity, colored by the hour of the day (night blue, morning orange, midday yellow, evening red), over a shaded box for the comfortable ranges. The title says how many of the hours were inside the box, and the corners name what lies that way: warm and dry, warm and humid, cool and dry, cool and humid.

## Usage

```
plot_comfort(
  temperature,
  humidity,
  mode = "light",
  temperature_range = c(20, 25),
  humidity_range = c(40, 60)
)
```

## Arguments

| | |
|---|---|
| `temperature`, `humidity` | Data frames from `get_hourly()` of temperature and humidity. |
| `mode` | `"light"` or `"dark"`, the mode of the app. |
| `temperature_range`, `humidity_range` | The comfortable ranges, the same as the green bands of the Current tab: 20 to 25 degrees Celsius and 40 to 60%. |

## Value

A plotly object.

## Examples

```
hours <- expand.grid(hour = 0:23, date = as.Date("2026-10-01") + 0:6)
temperature <- transform(hours, value = 22 + 4 * sin((hour - 9) / 24 * 2 * pi))
humidity <- transform(hours, value = 45 - 8 * sin((hour - 9) / 24 * 2 * pi))
plot_comfort(temperature, humidity)
plot_comfort(temperature, humidity, mode = "dark")
```

## plot_daily_means *Bars of the daily averages of a particle sensor*

### Description

One bar per day, with the guideline of the World Health Organization for the average of a day as a dashed line, so the days above it stand out. The axis starts at 0 and always reaches above the guideline.

### Usage

```
plot_daily_means(daily, variable, guideline, mode = "light")
```

### Arguments

| | |
|---|---|
| daily | Data frame from get_daily(). |
| variable | Name of the sensor, for the title and the hover labels. |
| guideline | The guideline for the average of a day. |
| mode | "light" or "dark", the mode of the app. |

### Value

A plotly object.

### Examples

```
daily <- data.frame(date = as.Date("2026-10-01") + 0:29, value = 8 + 10 * abs(sin(1:30)), hours = 24)
plot_daily_means(daily, "PM2.5", 15)
url <- "http://pip1.crabdance.com:1507/api/v1/query_range"
plot_daily_means(get_daily(url, "PM25", days = 30), "PM2.5", 15)
```

## plot_daily_pattern *Heatmap of the hourly averages of one sensor*

### Description

A column per day and a row per hour of the day, from midnight at the top, each colored by the average of that hour, so that what happens at the same time every day shows as a horizontal band. The colors stop at the 2nd and 98th percentiles, so that a few spikes do not wash out the rest; the hover label has the exact value.

### Usage

```
plot_daily_pattern(hourly, variable, unit = "", mode = "light", mid = NULL)
```

### Arguments

| | |
|---|---|
| hourly | Data frame from get_hourly(). |
| variable | Name of the sensor, for the title and the hover labels. |
| unit | Unit of the readings, "" for none. |
| mode | "light" or "dark", the mode of the app. |
| mid | A value to color white, with lower values in blue and higher values in red, such as room temperature; the color range is then made symmetric around it. NULL (the default) for one blue, from light to dark. |

### Value

A plotly object.

### Examples

```
hourly <- expand.grid(hour = 0:23, date = as.Date("2026-10-01") + 0:13)
hourly$value <- 21 + 2 * sin((hourly$hour - 9) / 24 * 2 * pi)
plot_daily_pattern(hourly, "Temperature", "C")
plot_daily_pattern(hourly, "Temperature", "C", mid = 24)
url <- "http://pip1.crabdance.com:1507/api/v1/query_range"
plot_daily_pattern(get_hourly(url, "temperature", days = 30), "Temperature", "C", mid = 30)
plot_daily_pattern(get_hourly(url, "temperature", days = 360), "Temperature", "C", mid = 30)
```

---

## plot_gauge *Plot Gauge*

---

### Description

Plots the last reading of one sensor as a gauge, with the ideal range in green and a short note on what the gas and particle sensors measure. The ideal ranges of PM2.5 and PM10 are the 24-hour guidelines of the WHO (2021); PM1, which has none, gets that of PM2.5. The gas and proximity sensors have no fixed range, so their gauge is scaled to all the readings of data: it spans the lowest to the highest reading, widened by one standard deviation, the green band is the mean plus or minus one standard deviation, and the delta is the difference from the mean.

### Usage

```
plot_gauge(data, variable = "Temperature")
```

### Arguments

| | |
|---|---|
| data | data frame from get_data(); the gauge shows its last reading |
| variable | one of "Temperature" "Humidity" "Pressure" "Lux" "Proximity" "NH3" "Reducing" "Oxidising" "PM1" "PM2.5" "PM10" |

## Value

A plotly object.

## Examples

```
prometheus_url <- c(
  "http://pip1.crabdance.com:1507/api/v1/query_range",
  "http://pip1.crabdance.com:1505/api/v1/query_range"
)
filtered_df <- get_data(urlq = prometheus_url[1])
plot_gauge(data = filtered_df, variable = "Temperature")
plot_gauge(data = filtered_df, variable = "Humidity")
plot_gauge(data = filtered_df, variable = "PM2.5")
```

---

# plot_time_series *Plot Time Series*

---

## Description

Plots the readings of one sensor, with their moving average, their mean, and a summary of the last reading, mean, minimum, maximum, standard deviation, range and dates on the right.

## Usage

```
plot_time_series(data, variable = "Temperature", scale = "", k = 30)
```

## Arguments

| | |
|---|---|
| data | data frame from get_data() |
| variable | one of "Temperature" "Humidity" "Pressure" "Lux" "Proximity" "NH3" "Reducing" "Oxidising" "PM1" "PM2.5" "PM10" |
| scale | title of the y axis |
| k | integer width of the rolling window. Must be odd for rollmedian |

## Value

A plotly object.

## Examples

```
prometheus_url <- c(
  "http://pip1.crabdance.com:1507/api/v1/query_range",
  "http://pip1.crabdance.com:1505/api/v1/query_range"
)
filtered_df <- get_data(urlq = prometheus_url[1])
plot_time_series(data = filtered_df, variable = "Temperature")
plot_time_series(data = filtered_df, variable = "Humidity")
plot_time_series(data = filtered_df, variable = "Pressure")
plot_time_series(data = filtered_df, variable = "PM2.5")
```

## red_steps *Steps of one red, from light to dark*

### Description

The red counterpart of blue_steps, with the same lightness at each step, for scales where higher means hotter or worse. Reverse it with `rev(red_steps)` in dark mode, as `plot_daily_pattern()` does with blue.

### Usage

```
red_steps
```

### Format

A character vector of 5 hexadecimal colors.

### Examples

```
red_steps
# as a plotly color scale
lapply(seq_along(red_steps), function(i) list((i - 1) / 4, red_steps[i]))
```

## symbol_mean *Greek mu and sigma, degrees Celsius, and micrograms per cubic metre*

### Description

Written as code points, as the R code of a package has to stay ASCII.

### Usage

```
symbol_mean

symbol_sd

degrees_celsius

micrograms_m3
```

### Format

Character strings.

### Examples

```
paste0("22 ", degrees_celsius, ", ", symbol_mean, " = 12 ", micrograms_m3)
```

`who_guidelines` *The guidelines of the World Health Organization for particles*

## Description

The air quality guidelines of 2021, in micrograms per cubic metre: for the average of a day, which should not be exceeded on more than 3 to 4 days a year (the 99th percentile), and for the average of a year. PM1 has none.

## Usage

```
who_guidelines
```

## Format

A data frame with the columns `metric` (`"PM25"`, `"PM10"`), `day` and `year`, in micrograms per cubic metre.

## Examples

```
who_guidelines
```

# Index


∗ **gauge**
    plot_gauge, 11
∗ **plot**
    plot_gauge, 11
    plot_time_series, 12
∗ **prometheus**
    get_data, 5
∗ **query**
    get_data, 5
∗ **timeseries**
    plot_time_series, 12

app_time_zone, 2

blue_steps, 3, *6*, *13*

comfort_summary, 3

degrees_celsius *(*symbol_mean*)*, 13

get_daily, 4
get_daily(), *10*
get_data, 5
get_hourly, 6
get_hourly(), *3*, *8*, *9*, *11*
golem::with_golem_options(), *7*

metric_mid, 6
micrograms_m3 *(*symbol_mean*)*, 13

pimoroni, 7
pimoroni(), *2*
plot_average_day, 8
plot_comfort, 9
plot_daily_means, 10
plot_daily_pattern, 10
plot_daily_pattern(), *6*, *13*
plot_gauge, 11
plot_time_series, 12

red_steps, 13

shiny::shinyApp(), *7*, *8*
symbol_mean, 13
symbol_sd *(*symbol_mean*)*, 13

who_guidelines, *6*, 14